

# **Aula Completa**

**Publicando com EAS — App Store e Play Store**

# Objetivos da Aula

Ao final desta aula, o aluno será capaz de:

- Entender o que é o **EAS (Expo Application Services)**
- Configurar o arquivo `eas.json` e o `app.json`
- Gerar builds para **Android** e **iOS** na nuvem
- Publicar na **Google Play Store** e na **Apple App Store**
- Entender o fluxo de credenciais e assinatura de apps

## O que é EAS?

**EAS** é o serviço de build e deploy oficial do Expo.

Com ele você consegue:

- Compilar seu app na nuvem (sem precisar de Mac/Android Studio local)
- Gerar o `.apk` , `.aab` (Android) e o `.ipa` (iOS)
- Enviar diretamente para as lojas com `eas submit`
- Publicar atualizações OTA com `eas update`

**OTA = Over The Air:** atualiza o JS sem precisar publicar nova versão na loja

## Comparativo: Expo Go vs EAS Build

| Característica        | Expo Go    | EAS Build             |
|-----------------------|------------|-----------------------|
| Execução rápida       | ✅ Sim      | ❌ (build demora)      |
| Plugins nativos       | ❌ Limitado | ✅ Suporte completo    |
| Publicação na loja    | ❌ Não      | ✅ Sim                 |
| Precisa de Mac p/ iOS | ❌ Não      | ❌ Não (nuvem do Expo) |
| APK/IPA real          | ❌ Não      | ✅ Sim                 |

## Pré-requisitos

Antes de começar, você precisa:

1. **Conta no Expo** → [expo.dev](https://expo.dev)
2. **Conta na Apple Developer** → [developer.apple.com](https://developer.apple.com) (paga, ~\$99/ano)
3. **Conta no Google Play** → [play.google.com/console](https://play.google.com/console) (paga, \$25 única vez)
4. Node.js instalado
5. EAS CLI instalado

## Instalando o EAS CLI

```
# Instalar o EAS CLI globalmente  
npm install -g eas-cli
```

```
# Verificar a versão  
eas --version
```

```
# Fazer login com sua conta Expo  
eas login
```

Após o login, confirme com:

```
eas whoami  
# Deve exibir o seu usuário Expo
```

# ⚙️ Configurando o app.json

O `app.json` é o coração da configuração do seu app Expo.

```
{
  "expo": {
    "name": "Meu App",
    "slug": "meu-app",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "splash": {
      "image": "./assets/splash.png",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    }
  }
}
```

# ⚙️ app.json — Configurações Android

```
{
  "expo": {
    "android": {
      "package": "com.empresa.meuapp",
      "versionCode": 1,
      "adaptiveIcon": {
        "foregroundImage": "./assets/adaptive-icon.png",
        "backgroundColor": "#ffffff"
      },
      "permissions": ["CAMERA", "ACCESS_FINE_LOCATION"]
    }
  }
}
```

- `package` : identificador único no formato **reverse domain** (ex: `com.empresa.meuapp` )
- `versionCode` : número inteiro que aumenta a cada publicação



# ⚙️ app.json — Configurações iOS

```
{
  "expo": {
    "ios": {
      "bundleIdentifier": "com.empresa.meuapp",
      "buildNumber": "1",
      "infoPlist": {
        "NSCameraUsageDescription": "Usamos a câmera para fotos do perfil.",
        "NSLocationWhenInUseUsageDescription": "Usamos a localização para mapas."
      }
    }
  }
}
```

- `bundleIdentifier`: igual ao `package` do Android — deve ser único na App Store
- `buildNumber`: string que aumenta a cada envio para a Apple
- `infoPlist`: mensagens obrigatórias para cada permissão solicitada

## Inicializando o EAS no projeto

Na raiz do projeto, execute:

```
eas build:configure
```

Isso cria automaticamente o arquivo `eas.json` com os perfis de build.



# Entendendo o eas.json

```
{
  "cli": {
    "version": ">= 12.0.0"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {}
  },
  "submit": {
    "production": {}
  }
}
```

## Perfis do eas.json

| Perfil      | Uso                                               |
|-------------|---------------------------------------------------|
| development | Dev Client — testes locais com plugins nativos    |
| preview     | APK/IPA interno para testes em dispositivos reais |
| production  | Build final para publicação nas lojas             |



# Build para Android

```
# Build de produção para Android (gera .aab)
eas build --platform android --profile production

# Build de preview para Android (gera .apk para testar)
eas build --platform android --profile preview
```

O resultado é um arquivo `.aab` (Android App Bundle) para produção ou `.apk` para testes.

- O `.aab` é exigido pela Play Store desde 2021.
- O `.apk` serve apenas para testes internos.

# Build para iOS

```
# Build de produção para iOS (gera .ipa)
eas build --platform ios --profile production
```

O EAS vai perguntar se você quer que ele gerencie as **credenciais automaticamente**.

Escolha **Yes** (recomendado para iniciantes) — o Expo vai:

- Criar o certificado de distribuição
- Criar o provisioning profile
- Registrar o Bundle Identifier na Apple

Você precisa estar logado com sua Apple ID de desenvolvedor.

## Credenciais — Android

No Android, a assinatura do app é feita com uma **keystore**.

Com EAS, você pode:

```
# Deixar o EAS gerar e armazenar a keystore (recomendado)
eas credentials
```

**ATENÇÃO:** Nunca perca sua keystore!

Sem ela, você não consegue atualizar seu app na Play Store.

O EAS armazena com segurança na nuvem se você permitir.

## Credenciais — iOS

No iOS, você precisa de:

1. **Apple Developer Account** ativa
2. **Certificado de Distribuição** (Distribution Certificate)
3. **Provisioning Profile** de distribuição
4. **App ID** registrado no Apple Developer Portal

O EAS gerencia tudo isso automaticamente com:

```
eas credentials --platform ios
```





# Configurando a Google Play Console

Passos para publicar no Android:

1. Acesse [play.google.com/console](https://play.google.com/console)
2. Crie um **novo app**
3. Preencha: nome, idioma, tipo (app/jogo), gratuito/pago
4. Aceite as **declarações de conformidade**
5. Vá em **Configuração > App integrity** e vincule sua chave de upload
6. Acesse **Versões > Produção** e faça upload do `.aab`



# Trilha de publicação — Play Store

Criação do App



Configurações da loja (ícone, capturas, descrição)



Política de privacidade



Classificação indicativa (questionário)



Upload do AAB (Produção > Nova versão)



Revisão (minutos a horas)



Publicado

# Configurando a App Store Connect

Passos para publicar no iOS:

1. Acesse [appstoreconnect.apple.com](https://appstoreconnect.apple.com)
2. Clique em + e crie um novo App
3. Informe: plataforma (iOS), nome, Bundle ID, SKU
4. Preencha as informações do app:
  - Capturas de tela (obrigatório para cada tamanho de dispositivo)
  - Descrição, palavras-chave, URL de suporte
5. Envie o build via EAS ou Xcode

# Trilha de publicação — App Store

Criação do App no App Store Connect



Envio do build (eas submit ou Transporter)



Informações de metadados (descrição, capturas)



Classificação etária (questionário IARC)



Política de privacidade (URL obrigatória)



Revisão da Apple (1 a 3 dias)



 Publicado

## **eas submit** — enviando para as lojas

O EAS também pode enviar o build direto para as lojas:

```
# Enviar para o Android (Google Play)
eas submit --platform android --profile production

# Enviar para o iOS (App Store)
eas submit --platform ios --profile production
```

Para o Android, você precisa de uma **chave de serviço JSON** da Play Console.  
Para o iOS, você precisa de uma **Apple API Key** (chave de App Store Connect).

# Google Play — Chave de Serviço JSON

Para o `eas submit` funcionar com Android:

1. Acesse a **Play Console** → **Configuração** → **Acesso à API**
2. Vincule seu projeto ao **Google Cloud**
3. Crie uma **conta de serviço** com permissão de **Release Manager**
4. Baixe o arquivo `service-account.json`
5. Informe no eas.json:

```
"submit": {  
  "production": {  
    "android": {  
      "serviceAccountKeyPath": "./service-account.json",  
      "track": "production"  
    }  
  }  
}
```

# Apple — App Store Connect API Key

Para o `eas submit` funcionar com iOS:

1. Acesse **App Store Connect** → **Usuários e Acesso** → **Chaves**
2. Gere uma nova chave com permissão de **App Manager**
3. Baixe o arquivo `.p8` (só pode baixar uma vez!)
4. Guarde o **Key ID** e o **Issuer ID**
5. Informe no eas.json:

```
"submit": {  
  "production": {  
    "ios": {  
      "appleId": "seu@email.com",  
      "ascAppId": "1234567890",  
      "appleTeamId": "ABCDE12345"  
    }  
  }  
}
```



## Atualizações OTA com eas update

Com o EAS Update você pode publicar correções de JS **sem passar pela revisão da loja**:

```
# Instalar o pacote necessário  
npx expo install expo-updates  
  
# Publicar uma atualização OTA  
eas update --branch production --message "Corrige bug no login"
```

O app verifica atualizações disponíveis ao iniciar.  
Funciona apenas para mudanças em JavaScript — não para código nativo.

# Checklist Pré-publicação

Antes de submeter, verifique:

- ☐ `version` e `versionCode` / `buildNumber` atualizados
- ☐ Ícone do app (1024x1024 para iOS, 512x512 para Android)
- ☐ Splash screen configurada
- ☐ `package` / `bundleIdentifier` corretos e únicos
- ☐ Permissões justificadas no `infoPlist` (iOS)
- ☐ Política de privacidade publicada em uma URL pública
- ☐ Capturas de tela de diferentes tamanhos
- ☐ Descrição do app traduzida (pt-BR e en-US)

## Erros Comuns

| Erro                         | Causa                                     | Solução                                                     |
|------------------------------|-------------------------------------------|-------------------------------------------------------------|
| Bundle ID já existe          | ID em uso por outro dev                   | Escolha outro <code>bundleIdentifier</code>                 |
| Version code duplicado       | <code>versionCode</code> não incrementado | Aumente o <code>versionCode</code> no <code>app.json</code> |
| Missing privacy policy       | URL de privacidade não definida           | Crie e informe a URL da política                            |
| Invalid provisioning profile | Credencial iOS desatualizada              | Rode <code>eas credentials</code> novamente                 |
| Keystore not found           | Keystore Android perdida                  | Use <code>eas credentials</code> para recuperar             |

# Fluxo Completo — Do Código à Loja

Código React Native (Expo)



`eas build (nuvem)`



Build gerado (.aab / .ipa)



`eas submit`



Play Console



App Store Connect



Revisão



Revisão



✓ Play Store



✓ App Store

# Resumo Final

| <b>Etapas</b>         | <b>Comando / Local</b>                      |
|-----------------------|---------------------------------------------|
| Instalar EAS CLI      | <code>npm install -g eas-cli</code>         |
| Login                 | <code>eas login</code>                      |
| Configurar projeto    | <code>eas build:configure</code>            |
| Build Android         | <code>eas build --platform android</code>   |
| Build iOS             | <code>eas build --platform ios</code>       |
| Submit para lojas     | <code>eas submit --platform android</code>  |
| Atualização OTA       | <code>eas update --branch production</code> |
| Gerenciar credenciais | <code>eas credentials</code>                |

## Conclusão

Com o **EAS (Expo Application Services)** você tem:

- Um fluxo moderno e simplificado de publicação mobile
- Build na nuvem sem precisar de Mac
- Gerenciamento automático de credenciais e assinaturas
- Submissão direta para **Google Play** e **App Store**
- Atualizações OTA sem passar pela revisão das lojas

O EAS é hoje o padrão recomendado para qualquer app Expo que vai para produção.